FREERTRTOS AUTOPILOT

Building a Custom Autopilot

A compact reference for turning the demo-filetree-lattice multicopter scaffold into a custom autopilot.

Version 0.1.0 · 31 October 2026 · Status: scaffold, not flight-ready

Reference target: THEJAS32 (RV32) · Runtime: FreeRTOS + freertrtos · GCS: MAVLink 2

This guide explains what the scaffold provides, which contracts to preserve, and the order in which to implement a custom multicopter autopilot. It is intentionally compact: the `docs/` folder contains the detailed module documentation, while this guide is the build roadmap.

> **Safety notice.** The repository is a scaffold, not flight-ready software. Most module task loops, driver transactions and persistence paths are TODO stubs. Do not fly until every module is implemented, unit-tested, exercised in SITL/HIL and validated in a controlled flight-test campaign. Props off for all bench work; keep an independent hardware kill switch.

How to use this guide

- **Evaluating the scaffold:** sections 1–3.
- **Porting to your hardware:** sections 4–5 and recipe 6.5.
- **Implementing the flight stack:** sections 4, 6 and 7.
- **Preparing to fly:** sections 8–10.

Contents

1. What this repository is 2
2. Architecture 2
3. Implementation status 3
4. Contracts to preserve 4
5. Build and run 6
6. Customization recipes 7
7. Implementation order 9
8. Verification and safety 9
9. Reference 10
10. Readiness checklist 11

1. What this repository is

This is a FreeRTOS-based multicopter autopilot scaffold laid out after the PX4-Autopilot source tree. It gives you the file tree, interfaces, task boundaries, configuration format and build wiring. It does not yet give you a flying vehicle: most flight tasks are explicit stubs, and several drivers stop at register-level TODOs.

Item	Value
Project	freertrtos_autopilot 0.1.0
Vehicle	Multicopter, quad-X reference airframe
Architecture	PX4-inspired layers with uORB-style pub/sub
Operating system	FreeRTOS through the freertrtos adaptation layer
Reference board	THEJAS32 RISC-V; POSIX SITL board shim
Actuation	PWM/DShot, quad-X mixer and actuator limits
Telemetry	MAVLink 2; QGroundControl / Mission Planner intent
Build	CMake + Makefile; host unit tests and firmware
Status	Scaffold — not flight-ready

Table 1. Project at a glance.

The central rule is simple: flight modules exchange data through typed uORB topics; they do not call each other's control functions. MAVLink is a peer of the flight stack, never part of the control loop. The commander owns arming, mode state and failsafe actions.

> **What it is not.** It is not a PX4 fork, a drop-in ArduPilot replacement, a certified flight controller, or an autopilot you can flash and fly today. It is a chassis for a builder who knows embedded C, FreeRTOS and the hardware being used.

2. Architecture

Dependencies point downward only. A driver never includes a flight module, and a module never pokes a register directly. Each stage in the control path reads a topic snapshot and publishes a new topic.

```
GCS / Mission Planner --> mavlink ------+
                                        |
on-board shell --------> systemcmds ----+--> flight modules

sensors --> state_estimator --> flight_mode_manager
              |                         |
              v                         v
         navigator                mc_pos_control
                                        |
                                        v
                                  mc_att_control
                                        |
                                        v
                                  control_allocator
                                        |
                                        v
                                  pwm_out / dshot --> ESCs

middleware: uORB · params · time · storage · logging · work queues
drivers:    IMU · baro · mag · GNSS · RC · PWM · telemetry · battery · SD
HAL -> BSP (thejas32) -> freertrtos -> FreeRTOS kernel + port
```

Control data flow: (1) sensors publish IMU, baro, magnetometer, GNSS, RC and battery samples; (2) the state estimator publishes attitude, angular velocity and local/global position; (3) commander and the flight-mode manager select the active mode and produce a trajectory or rate setpoint; (4) position control converts the trajectory setpoint into an attitude setpoint; (5) attitude and rate control produce torque and thrust; (6) the control allocator mixes those into four motor commands; (7) PWM/DShot drives the ESCs.

The seams matter more than the current implementations. You can replace the estimator, a controller, the mixer or the GCS layer without rewriting the rest, as long as the topic layouts and timing contracts stay intact. That also makes each stage testable and loggable in isolation.

2.1 Mapping back to PX4

PX4 concept	This project	Purpose
msg/ + uORB	src/msg + src/middleware/uorb	Typed asynchronous data bus
src/drivers	src/drivers	Sensor, actuator and telemetry drivers
sensors module	src/modules/sensors	Acquisition, calibration, timestamping
ekf2	src/modules/state_estimator	Attitude and position estimation
commander	src/modules/commander	Arming, modes, failsafe, home
flight_mode_manager	src/modules/flight_mode_manager	Mode logic and setpoints
mc_pos/att_control	src/modules/mc_*_control	Position, attitude and rate control
control_allocator	src/modules/control_allocator	Torque/thrust to motors
navigator	src/modules/navigator	Mission, RTL, waypoint sequencing
mavlink + logger	src/modules/mavlink, logger	GCS protocol and flight logs
ROMFS rcS / boards	config/boards, boards	Boot script and board support

2.2 Where things live

Path	Role
src/main	Boot, module registry, app timing and priorities
src/msg	32 message definitions and generated structs
src/middleware	uORB, params, time, storage, logging, work queue
src/drivers	IMU, baro, mag, GNSS, RC, PWM, telemetry, battery, SD
src/modules	Sensors, estimator, commander, modes, controllers, navigator, MAVLink, logger
src/lib	Math, geodesy, mission, RTL, CRC, ring buffer, atmosphere
src/hal + boards	HAL interfaces and board support packages
config	Airframes, parameters, missions, failsafe and stream policy
tests	Host unit tests; SITL and HIL plans
tools	Message generator, flashing, MAVLink scripts, log decoder

3. Implementation status

The repository is uneven on purpose: the pure logic and protocol pieces are implemented and tested; the real-time glue is mostly left for the integrator. Use the table below as a work breakdown, not a scorecard.

Area	State	What exists	What you add
Build and tests	Working	CMake host/firmware build, CI, 17 host test groups	Target toolchain and board checks
Messages	Working	32 .msg files, generator, fixed-size structs	Your custom topics and log schemas
Libraries	Implemented	vec/quat/PID, geodesy, 64-item mission, RTL state machine	Tuning, extra mission commands
Control and mixer	Implemented	Attitude, rate, position control, quad-X mixer, limits	Task wiring, gains, new airframes
MAVLink codec	Implemented	Parser, encoder, messages, commands, mission upload	Transport and task loop
Log format	Implemented	Binary header/records, CRC, topic table, log2csv	Logger task and storage integration
Drivers	Stubs	Device files and HAL contracts	Bus transactions, calibration, health
Middleware	Partial	uORB registry, parameter table, time	Flash persistence, storage and log writer
Module tasks	Stubs	Start/stop files, mode logic, guidance helpers	Task loops and subscriptions
SITL / HIL	Planned	POSIX board shim, test plans	Plant model, sensor sim, HIL rig

- **Trust the tested parts.** The control math, mission logic, MAVLink framing and log format already have host tests. Keep those tests green when you change them.

- **Expect to write the real-time layer.** Every module `*_start()` currently logs a scaffold warning and returns `-1`, except the helper code inside it.
- **Wire the missing modules.** `app_init()` starts ten modules, but the flight-mode manager, position/attitude controllers and control allocator are not in its list yet. Add them when their tasks exist.
- **Do not trust persistence yet.** `params_save()` is a TODO that returns `-1`; the logger task and storage path are also stubs.

Definition of done for a module. Interface documented in the header; unit tests for pure logic; a SITL scenario if it affects flight; failsafe behaviour defined; parameters documented; logged topics identified; no dynamic allocation in the periodic path; stack and CPU budget measured.

4. Contracts to preserve

You can replace almost every implementation in this tree, but these contracts are the load-bearing walls. Break them and the modules no longer compose.

4.1 uORB topics

All inter-module data moves through the static uORB registry. Topics hold fixed-size, last-value payloads; a publish copies the payload under a short lock and a copy returns the latest snapshot. The limits are 64 topics and 256 bytes per payload.

- Every message starts with `uint64 timestamp` and is defined in `src/msg/*.msg`.
- Run `make msg` (or `tools/msg_gen.py`) after changing a message; generated `*_s` structs live in `src/msg/generated/uorb_topics.h`.
- Append fields at the end. Never insert a field in the middle: the logger and GCS mapping assume a stable layout.
- Do not pass pointers between tasks. Payloads are copied; keep them fixed-size and allocation-free.
- Use `uorb_topic()`, `uorb_publish()`, `uorb_copy()`, `uorb_wait()` and `uorb_sequence()`; do not create a second bus.

Family	Topics
Sensors	sensor_imu, sensor_baro, sensor_mag, sensor_gps, rc_channels, battery_status
Estimation	vehicle_attitude, vehicle_angular_velocity, vehicle_local_position, vehicle_global_position, estimator_status
Control	trajectory_setpoint, vehicle_attitude_setpoint, vehicle_rates_setpoint, vehicle_torque_setpoint, vehicle_thrust_setpoint, actuator_motors
System	vehicle_status, actuator_armed, failsafe_flags, battery_status, telemetry_status
Mission	mission_item, mission_current, home_position
GCS / logging	vehicle_command, vehicle_command_ack, mavlink_log, log_status, parameter_update

4.2 Timing and tasks

The timing budget lives in `src/main/app_config.h`. Treat the values below as the contract when you create tasks. FreeRTOS uses higher numeric priority for higher urgency; the assignment is rate-monotonic.

Task	Rate	Priority	Stack (words)	Role
rate control	1000 Hz	24	1024	Inner rate loop
attitude control	400 Hz	23	1024	Attitude to rate
position control	100 Hz	22	1024	Position to attitude
sensors	400 Hz	20	1024	Acquire and publish samples
state estimator	200 Hz	19	2048	Attitude and position
commander	50 Hz	18	1536	Arming, modes, failsafe
navigator	50 Hz	17	2048	Mission and RTL guidance
mavlink	50 Hz	12	3072	GCS transport and protocol
logger	50 Hz	8	2048	Buffered log writes
monitor	1 Hz	4	512	Health and housekeeping

- The watchdog timeout is 500 ms. Every periodic task should check in; a missed deadline becomes a failsafe, not a silent stall.
- No dynamic allocation, unbounded blocking, or SD/UART I/O in a control loop. Logging may drop records; it may not back-pressure flight code.
- Stack sizes are in words (4 bytes on RV32), not bytes. Measure high-water marks over a full mission before reducing them.
- Create tasks in the module `*_start()` function, publish their health, and add them to the boot module registry in `src/main/init.c`.

4.3 Parameters

Parameters are float or int32 values with names of at most 16 characters. The defaults live in `src/middleware/params/param_table.c`; keep `config/boards/thejas32/default.params` in sync. Read a parameter once during initialisation, not inside a control loop.

Use consistent prefixes: `COM_` for commander and safety, `BAT_` for battery, `MC_` for attitude/rate control, `MPC_` for position control, `NAV_` for navigation, `RTL_` for return-to-launch, `MAV_` for the GCS link and `LOG_` for logging. `params_save()` must be implemented with a CRC-protected flash write before you rely on a value surviving a reboot.

4.4 Arming and failsafe

Commander owns the arming state machine and the failsafe action. Detectors publish evidence (`estimator_status`, `battery_status`, `telemetry_status`, `failsafe_flags`); they never command motors directly. Pre-arm checks include attitude validity, gyro/accel/mag calibration, GNSS readiness for position modes, battery voltage, RC presence and throttle-low, no active failsafe, a loaded mission when required, and the safety switch or GCS authorisation.

The action hierarchy is `NONE < HOLD < RTL < LAND < DISARM`. The most severe active action wins, and it latches until the trigger clears and the pilot explicitly re-selects a mode. The default policy is in `config/failsafe/failsafe.conf`; `commander/failsafe.c` implements the severity ordering.

Trigger	Default action	Notes
Battery low	RTL	Filtered voltage; hysteresis on recovery
Battery critical	LAND	Never auto-clears in flight
RC loss	RTL	Timeout, then re-arm required
GCS loss	HOLD / continue	Configurable; continue mission before RTL
GNSS loss	HOLD	Fall back to altitude hold, then RTL if safe
Estimator invalid	LAND	Attitude fallback only if valid
Mission abort	RTL	Hold briefly, then return
Watchdog overrun	LAND or reboot	Task health is a safety input
Kill switch	DISARM	Hardware path; never overridable

4.5 Modes and missions

The internal modes are manual, stabilized, altitude hold, position hold, mission, RTL and land. MAVLink `custom_mode` is the internal mode index plus one, so a GCS mode mapping must use the same table. A mission holds at most 64 items and supports takeoff, waypoint, loiter, land, RTL and delay. The navigator owns the mission cursor; the flight-mode manager consumes only its target and status.

5. Build and run

5.1 Host unit tests

The host build needs no hardware, cross-compiler or kernel checkout. It is the fastest gate and should stay green on every change.

```
cd /home/adithyan/demo-filetree-lattice
make test

# equivalent:
cmake -S . -B build/host -DFREERTRTOS_BUILD_FIRMWARE=OFF \
      -DFREERTRTOS_BUILD_TESTS=ON
cmake --build build/host --parallel
ctest --test-dir build/host --output-on-failure
```

The suite currently covers vector math, quaternions, PID, geodesy, missions, atmosphere, RTL, attitude/rate/position control, the quad-X mixer, actuator limits, log format, MAVLink framing, command mapping, mission upload and waypoints. A passing run ends with `100% tests passed` and `all tests passed`.

5.2 Firmware build

Requirement	Notes
CMake 3.20+	Host and firmware configuration
RISC-V GCC	Reference prefix `riscv32-unknown-elf-`
FreeRTOS-Kernel 10.5+	`third_party/FreeRTOS-Kernel`
THEJAS32 port	`third_party/freertos-thejas32`
Python 3.8+	Message generator and log/MAVLink tools
MAVLink C library	Optional; `third_party/mavlink`

```
cd third_party
git clone https://github.com/FreeRTOS/FreeRTOS-Kernel.git
git clone https://gitlab.com/riscv-vega/operating-systems/freertos-thejas32.git

cd ..
make thejas32
```

The build fails early with a clear message if the kernel or board port is missing. Artifacts are `build/thejas32/freertrtos_autopilot.elf` and `.map`. Convert to a raw binary with `riscv32-unknown-elf-objcopy -O binary`, and fill in the programmer command in `tools/flash.sh` before flashing. Never flash with props attached.

Build option	Default	Purpose
FREERTRTOS_BOARD	thejas32	Select the board package
FREERTRTOS_BUILD_FIRMWARE	ON	Build the target image
FREERTRTOS_BUILD_TESTS	ON	Build host unit tests
FREERTRTOS_KERNEL_DIR	third_party/FreeRTOS-Kernel	Kernel checkout
FREERTRTOS_PORT_DIR	third_party/freertos-thejas32	Port checkout
FREERTRTOS_WERROR	OFF	Treat warnings as errors

5.3 SITL and troubleshooting

The SITL board shim exists, but there is no plant model or sensor simulator checked in yet. The flight modules still return `-1` from their start functions, so an empty SITL run is not evidence of flight readiness. Build the simulation before relying on the SITL test plan.

Symptom	Likely cause
FreeRTOS kernel not found	Kernel checkout missing or path wrong
Could not find port.c	Port layout differs; set FREERTRTOS_PORT_DIR

Symptom	Likely cause
Undefined reference to _start	Startup file or linker script missing
Immediate reset after boot	Stack overflow, vector table or clock
configASSERT loop	Invalid object or interrupt priority
No PWM output	Timer/pin map, arming state or safety switch
No telemetry	UART pins/baud, system ID or GCS baud
Watchdog reboot in flight	Task overrun or ISR starvation

6. Customization recipes

These are the shortest safe paths for the changes a custom autopilot usually needs. Every recipe ends with a host test or a bench check; add the test before you wire the change into a task.

6.1 Add a message

Create a `.msg` file with a timestamp first, generate the C struct, then use the generated `*_s` type. Keep the payload under 256 bytes.

```
# src/msg/my_payload.msg
uint64 timestamp
float32 value
bool valid

make msg
# -> src/msg/generated/uorb_topics.h  (my_payload_s)
```

6.2 Publish, subscribe and run a task

A module creates its topics and task in `*_start()`, publishes from its own loop, and copies the latest snapshot on the consumer side. Use the app timing table for rate, priority and stack.

```
static uorb_topic_t *topic;

static void my_task(void *arg)
{
    my_payload_s sample = {0};
    (void)arg;

    for (;;) {
        sample.timestamp = hrt_absolute_time();
        sample.value = read_my_sensor();
        sample.valid = true;
        (void)uorb_publish(topic, &sample);
        frt_delay_ms(2);   /* 500 Hz; use vTaskDelayUntil for tight jitter */
    }
}

int my_module_start(void)
{
    frt_task_t task;
    topic = uorb_topic("my_payload", sizeof(my_payload_s));
    if (topic == NULL) return -1;
    return frt_task_create("my_module", my_task, NULL,
                           1024, 18, &task) == FRT_OK ? 0 : -1;
}
```

6.3 Add a flight mode

A mode implements enter/update/exit and writes the mode context output. Add the enum, declare the ops structure, add the switch case in `flight_mode_manager.c`, then update the MAVLink custom-mode map.

```
static int my_mode_enter(mode_context_t *ctx)
{
    return ctx->attitude_valid ? 0 : -1;
}

static int my_mode_update(mode_context_t *ctx, float dt_s)
{
    (void)dt_s;
    ctx->output.roll   = mode_stick_to_angle(ctx->roll_stick, 0.61f);
    ctx->output.pitch  = mode_stick_to_angle(ctx->pitch_stick, 0.61f);
    ctx->output.yaw    = ctx->heading_rad;
    ctx->output.thrust = 0.5f * (ctx->thrust_stick + 1.0f);
    return 0;
}

const flight_mode_ops_t mode_my_ops = {
    .id = FLIGHT_MODE_MY, .name = "my_mode",
    .enter = my_mode_enter, .update = my_mode_update, .exit = NULL,
};
```

6.4 Support an airframe

- Create a mixer module in `src/modules/control_allocator/` with a `*_mix()` function and a unit test for motor order, signs, saturation and slew limits.
- Update `config/airframes/<name>/airframe.conf` and `mixer.conf` with motor count, positions, directions, arm length and gains.
- Call the mixer from `control_allocator.c`; load the airframe configuration once at init, not in the 1 kHz loop.
- Verify on a thrust stand with props removed and a current-limited supply before any flight attempt.

6.5 Port to a new board

1. Create `boards/<name>/` with `board.c`, `board.h`, `pinmap.h`, clock setup, `ld/link.ld` and `startup.S`.
2. Implement the HAL interfaces in `src/hal/include/hal/` under `boards/<name>/hal/`.
3. Add `config/boards/<name>/default.params` and `config/boards/<name>/init.d/rcS`.
4. Add the board to `cmake/options.cmake` and the Makefile, with a matching toolchain file.
5. Bring up in order: UART console, clock, GPIO, IMU, PWM (props off), RC, telemetry, GNSS, SD.
6. Measure tick cost, context-switch latency, stack high-water marks and control-loop jitter.

6.6 Add a parameter

Add the entry to the default table, mirror it in the board default file, and read it at init. Use a name of 16 characters or fewer.

```
/* src/middleware/params/param_table.c */
{"MY_GAIN", PARAM_TYPE_FLOAT, {.f = 1.0f}},

/* init */
my_gain = params_get_float("MY_GAIN", 1.0f);
```

6.7 GCS, mission and logging touchpoints

Change	Touchpoints
Add an outgoing stream	config/mavlink/streams.conf, mavlink_messages.c, mavlink_streams.c
Add a command	mavlink_commands.c, vehicle_command topic, COMMAND_ACK
Map a flight mode	mode.h, flight_mode_manager_get_ops(), custom_mode = id + 1
Upload a mission	mavlink_mission.c staging, lib/mission validation, storage
Change log topics	config/log/logger.conf, log_topics.c, tools/log2csv.py

Change	Touchpoints
Add a log field	message .msg, log format version, decoder schema

7. Implementation order

Build a stable hover before you build autonomy. The first milestone is not a waypoint mission; it is a restrained vehicle with trustworthy sensors, a working kill switch and a control loop that does not saturate or drift.

Phase	Goal	Primary work	Gate
0. Freeze	Define the target	Frame, sensors, GCS, power and safety concept	Written configuration and test plan
1. Platform	Boot and observe	BSP, HAL, clock, UART console, flash, watchdog	Banner, shell status and 60 s uptime
2. Infrastructure	Make data durable	uORB, params persistence, storage, log writer	Parameter survives reboot; log decodes
3. Drivers	Get real samples	IMU, baro, mag, GNSS, RC, battery, PWM, telemetry, SD	Stationary gyro < 0.5 °/s; accel ≈ 1 g; PWM 1000–2000 µs
4. Estimation	Know the state	Sensor aggregation, calibration, attitude and position	Attitude tracks; position valid in SITL
5. Control	Hold attitude	Attitude/rate/position control, mixer, limits	Thrust-stand step response; correct motor order
6. System safety	Make it safe	Commander, arming, modes, failsafe, RC, land detector	All pre-arm checks and injected failsafes pass
7. Autonomy	Follow guidance	Navigator mission/RTL, home, mission storage	Waypoint acceptance and RTL profile pass
8. GCS/logging	Observe and command	MAVLink transport/task, streams, commands, logger	QGC upload, telemetry and log analysis
9. Verification	Prove it	Host, SITL, HIL, bench and flight campaign	All gates signed off

A useful minimum path is phases 0–1 and 3–6, with only stabilized flight and a conservative altitude limit. Add barometer, GNSS, position control and mission guidance only after the stabilized hover is repeatable. MAVLink and logging can be added earlier for visibility, but they must never become prerequisites for stabilization.

> **Wiring note.** `app_init()` currently starts sensors, state estimator, commander, navigator, MAVLink, logger, battery status, RC update, land detector and the watchdog. Add `flight_mode_manager`, `mc_pos_control`, `mc_att_control` and `control_allocator` to the module registry when their `*_start()` functions create real tasks. Without them the control path is not connected, even if the libraries compile.

8. Verification and safety

The test pyramid is host unit tests, SITL missions, HIL/bench and controlled flight. Lower levels run on every change; higher levels run before a flight. SITL does not exercise real UART, SPI, PWM or timing, and it is not a substitute for HIL.

Level	What it proves	Minimum evidence
Host unit	Pure logic and protocol	`make test` passes; new code has tests
SITL	Full stack against a plant model	Hover, mission, RTL and failsafe scenarios
HIL / bench	Real board, real timing	Props off; logic analyser on outputs; 30 min soak
Flight	The complete system	Tether or clear area; kill switch; log every flight

Inject faults deliberately: stuck IMU, barometer drift, GNSS jump, RC loss, GCS loss, low battery, full or removed SD card, blocked task and stack overflow. A failsafe that has never been triggered is an assumption, not a feature.

8.1 First-flight gate

- Firmware hash and parameter set recorded; known-good recovery image kept.
- All host tests pass; SITL and HIL evidence reviewed.

- Props off: motor order, direction, PWM range and failsafe outputs verified.
- Accel, gyro, mag and baro calibrated; estimator health good.
- Arm/disarm from RC, GCS and shell; kill switch tested independently.
- Low battery, RC loss and GCS loss failsafes tested at safe altitude or on the bench.
- Log file written, survives power loss and decodes with `tools/log2csv.py`.
- Pilot briefed; flight area clear; altitude and distance limits agreed.

Safety rules. Props off for every bench test. Use a current-limited supply for the first ESC tests. Keep a hardware kill switch independent of the flight controller. Never disable a failsafe to make a test pass.

9. Reference

9.1 Configuration files

File	Controls
config/boards/thejas32/default.params	Board default parameters and starting gains
config/boards/thejas32/init.d/rcS	Intended boot sequence
config/airframes/quad_x/airframe.conf	Motor count, positions, directions, geometry
config/airframes/quad_x/mixer.conf	Logical mixer signs and limits
config/missions/demo_mission.plan	QGC mission plan
config/missions/demo_mission.waypoints	MAVLink WPL 110 mission
config/mavlink/streams.conf	Outgoing MAVLink message rates
config/log/logger.conf	Logged topics and rates
config/failsafe/failsafe.conf	Failsafe actions, timeouts and thresholds

9.2 Tools

Tool	Use
tools/msg_gen.py	Generate `uorb_topics.h` from `.msg` files
tools/gen_tree.py	Regenerate `docs/file-tree.md`
tools/log2csv.py	Decode `.ftlog` files to CSV and summary
tools/mavlink_status.py	Print live mode, position, battery and mission status
tools/mavlink_mission.py	Upload a QGC WPL 110 waypoint file
tools/flash.sh	Programmer template for the target board

9.3 Key defaults

Group	Parameters
System	MAV_SYS_ID=1, MAV_COMP_ID=1, MAV_BAUD=57600
Arming	COM_ARM_AUTH=1, COM_ARM_MAG=1, COM_ARM_GNSS=0
Timeouts	COM_RC_LOSS_TIMEOUT_MS=500, COM_GCS_LOSS_TIMEOUT_MS=5000
Battery	BAT_N_CELLS=4, BAT_LOW_VOLTAGE=3.5, BAT_CRIT_VOLTAGE=3.3
Attitude	MC_ROLL_P/MC_PITCH_P=6.0, MC_YAW_P=2.0
Rates	MC_ROLLRATE_P=0.15, MC_PITCHRATE_P=0.15, MC_YAWRATE_P=0.2
Position	MPC_XY_P=0.95, MPC_Z_P=1.0, MPC_XY_VEL_MAX=5.0
Navigation	NAV_ACC_RAD=2.0, NAV_LOITER_TIME=0.0
RTL	RTL_RETURN_ALT=30.0, RTL_DESCEND_ALT=10.0, RTL_LAND_ENABLE=1
Logging	LOG_LEVEL=4, LOG_MAX_FILE_MB=256, LOG_ON_ARM=1

These are safe starting points, not tuned flight gains. Validate every value on your airframe, and change one thing at a time.

9.4 Further reading

Document	Covers
docs/architecture.md	Layering, data flow and PX4 mapping
docs/task-schedule.md	Task rates, priorities and CPU budget
docs/flight-modes.md	Arming and mode behaviour
docs/failsafe.md	Failsafe matrix and recovery rules
docs/mavlink.md	Streams, commands and mission protocol
docs/logging.md	Log format, topics and post-flight tools
docs/build.md	Toolchain, memory map and flashing
docs/porting-guide.md	New board and driver porting
docs/testing.md	Host, SITL, HIL and flight test plan
docs/safety.md	Hazards, arming policy and limits

10. Readiness checklist

Copy this list into the project issue tracker and sign each line. An unticked box is an open risk, not a documentation gap.

Design and configuration

- Frame, motor order, prop direction, arm length and battery chemistry frozen.
- Sensor set and bus assignments documented; GCS and telemetry link chosen.
- Parameters mirrored between `param_table.c` and `default.params`.
- Airframe mixer, limits and failsafe policy reviewed by a second person.

Platform and estimation

- Board boots, console works, timebase is accurate and watchdog resets correctly.
- Parameters and missions survive reboot; storage and log writer are reliable.
- IMU, baro, mag, GNSS and RC produce plausible health and timing data.
- Estimator attitude, altitude and position valid through a full SITL/HIL run.

Control and safety

- Rate, attitude and position loops tuned on a thrust stand and in SITL.
- Arming checks and failsafes injected; recovery requires explicit pilot action.
- Kill switch cuts outputs independently of software.
- A stable stabilized hover is repeatable before any position-mode flight.

GCS, logging and release

- MAVLink streams, commands and mission upload verified with QGC/Mission Planner.
- Every flight logs attitude, position, status, motors, battery and mission progress.
- Firmware hash, parameter set, airframe config and flight log archived.
- Known-good recovery image and a written flight-test card are available at the field.

Release rule. Do not move to the next phase until the current gate has evidence: a test log, a bench measurement, a reviewed diff or a signed checklist. The scaffold rewards that discipline; it does not replace it.